

突击班-并发编程（第二天）

一、AQS是什么？（京东）

AQS本质就是JUC包下的一个抽象类，他本身是一个基础类，并没有太多的逻辑实现，更多的是提供了三个核心内容，让JUC包下的其他工具去继承AQS，实现具体的并发逻辑.....

```
public abstract class AbstractQueuedSynchronizer
```

AQS里有三个核心点：

核心属性state： 基于volatile修饰的int类型，并且在存在并发时，AQS也提供了基于CAS修改的方法。保证了原子性，可见性，有序性。

- ReentrantLock： state标识锁重入的次数。所谓的竞争锁资源，本质就是哪个线程成功的基于CAS将state属性从0改为1，就代表竞争锁资源成功。
- ReentrantReadWriteLock： state依然是表示锁资源的信息，读写锁种会将state拆分成两部分，高16bit位作为读锁资源，低16bit位作为写锁资源来使用。
- CountdownLatch： state表示具体的数值。计数器的具体数值
-

同步队列（双向链表）： 基于AQS中的内部类Node组成的一个同步队列，并且对这个队列的添加，移除Node都已经提供好了具体的API。

- 作用本质就是用来排队的，无论是锁，还是其他的工具类，只要是获取具体的资源的，并且没有获取到需要等待的情况，都需要将等待的线程封装为Node，扔到这个同步队列里排队。

单向链表： 也是基于AQS中的内部类Node，组成的单向链表，这里没有采用Node的prev，next指针，而是单独使用提供的nextWaiter组成的单向链表。依然采用Node是因为单向链表的Node可能迁移到同步队列中。

- 一般就用于锁，来实现类似synchronized中的wait、notify方法。也就是持有锁资源时，可以挂起线程，释放锁资源，等待被唤醒的操作。在AQS内部提供的是await、signal方法。

Ps：面试的聊的时候，需要举例，可以就针对lock锁来举例。

二、CAS是什么？（美团）

CAS本质就是Java中乐观锁的一种实现..... 比较和交换。

在Java中的Unsafe里，提供了执行CAS的API

```
# var1: 对象.....  
# var2: 对象里得哪个属性的内存偏移量  
# var4: oldValue, 旧值  
# var5: newValue, 新值  
public final native boolean compareAndSwapObject(Object var1, long var2,  
    Object var4, Object var5);
```

上面可以看到是native方法，走的C++，C++最后会调度到CPU的指令，cmpxchg。

CAS有人称他为无锁，不像synchronized、lock锁在没有竞争成功的情况下，会挂起线程。

CAS不会挂起线程，反馈一个结果，成功返回true，反之返回false，可以一直尝试.....（自旋锁）

Ps：无锁没问题，因为在Java层面不存在，但是说有锁也可以，因为在CPU层面有锁（缓存行锁、总线锁）

三、CAS的问题（爱奇艺）

CAS常见的问题有三个：

- CAS本质只是对某一个属性的修改实现原子性操作，他本质无法锁住一段代码。（但是，sync，lock锁可以锁住一段代码，但是他俩底层也是利用了CAS）
- ABA问题：
 - 如果只关注值本身，而不关注中间的修改过程，其实ABA并不是问题。因为值本身是没有变化的。
 - 如果既关注值本身，又关注值被修改的过程，ABA就存在问题了.....存在问题可以基于版本号来管理值的版本。JUC包下有提供解决ABA问题的类。

AtomicStampedReference

- 自旋次数过多：
 - CAS不会挂起线程，如果长时间执行CAS，并且不成功，会浪费CPU的资源。。。
 - 可以指定循环的次数，不是非要一直循环到成功，比如循环10次，如果不成功，挂起线程.....
 - 从另外的维度解决，如果业务可以支撑，仿照LongAdder的形式，提供多个资源去CAS。一般就适合（计数）

四、volatile（京东）（爱奇艺）

volatile本身是一个关键字，修饰属性的。

目的是为了这个属性保证 **可见性，有序性（禁止指令重排）**

可见性发生问题的原因，以及volatile存在的意义：

- 因为执行是由CPU去调度的，而CPU为了执行效率足够快，不会每次都从JVM内存中获取数据。
- 而是在CPU端，有缓存，L1，L2，L3，会将JVM中的数据缓存到L1~L3缓存里，每次操作都直接从缓存里获取。目的就是速度快。（L1-L2是CPU核心私有，L3是CPU核心共享）
- 如果CPU核心1，将JVM中的内存里得i = 1缓存到L1缓存中，此时CPU核心2对JVM中的i进行了修改，修改为了i = 2，此时就会出现，内核1 - i=1，内核2 - i=2。数据不一致，这就是不可见.....
- CPU将缓存的数据具体放到了L1缓存中的缓存行里。缓存行是CPU缓存里得最小单位。
- CPU本身就提供了一个多核CPU的缓存一致性的协议，MESI协议，解决了缓存行数据不一致的问题。
- But，MESI的触发会影响CPU的执行效率，CPU厂商会优化MESI协议，追加很多缓冲区。比如StoreBuffer，比如Invalid Queue。导致MESI协议出发不及时。
- volatile修饰的属性，在被操作时，会追加上一个 **lock前缀的指令**，而这个指令会强制出发MESI协议。最终就保证了可见性。

有序性：

- CPU在保证最终结果一致的前提下，为了提升效率，可能会将执行的指令做重排。
- 比如，创建对象，正常是1、开辟内存空间。2、初始化属性。3、基于对象地址引用。
- 但是，可能因为指令重排，导致前面正常的1、2、3的顺序，调整为了1、3、2。
- 这个问题就可以基于volatile解决，volatile修饰的属性，就会禁止CPU做指令重排。
- Ps：前面的创建对象例子，其实就是针对单例模式的懒加载中的DCL种，给对象追加volatile的原因。

五、线程池的工作原理、执行原理，执行过程.....（经常问.....）

说人话，就是投递任务到线程池之后，线程池是如何处理这个任务的。

前提，大家都掌握了线程池的7个核心参数。

corePoolSize, maxmumPoolSize, workQueue, reject, (keepaliveTime, unit, threadFactory)

下面这里是执行流程。

```
public void execute( @NotNull Runnable command) {
    if (command == null)
        throw new NullPointerException();
    /*...*/
    int c = ctl.get();
    if (workerCountOf(c) < corePoolSize) {
        if (addWorker(command, core: true))
            return;
        c = ctl.get();
    }
    if (isRunning(c) && workQueue.offer(command)) {
        int recheck = ctl.get();
        if (! isRunning(recheck) && remove(command))
            reject(command);
        else if (workerCountOf(recheck) == 0)
            addWorker( firstTask: null, core: false);
    }
    else if (!addWorker(command, core: false))
        reject(command);
}
```

1、工作线程个数少于核心线程数，创建核心线程去执行投递过来的任务

2、将任务扔到工作队列里排队，等待线程处理

3、工作队列满了，就会尝试创建非核心线程去处理投递过来的任务

4、核心够数了，工作队列满了，工作线程到最大线程数了，执行拒绝策略

六、线程池空闲线程的状态？线程池里任务过来怎么激活线程？（宇信科技）

线程池中空闲得线程可能有两种状态：

- WAITING：核心线程会执行take，长时间阻塞，直到获取到新任务。
- TIMED_WAITING：非核心线程会执行poll，等待最大空闲时间，如果期间没任务，线程销毁。

Ps：其实，线程池中的线程只有创建的时候区分核心与非核心，因为要判断不同的参数。不过创建出来启动之后，都是普通的工作线程。线程执行take还是执行poll，取决于工作线程 大于还是小于 corePoolSize，如果小于等于，执行take，如果大于，执行poll。

线程池里得任务如果投递到了工作队列，会基于工作队列的机制将线程唤醒。

本质是基于ReentrantLock唤醒的，在工作队列挂起时，会执行await方法，在被唤醒时，会执行signal方法，将线程唤醒。拿到任务去正常的执行。

七、线程池的参数和参数的作用（通力电梯）（阿里乌鸪）

线程池的这几个参数，必须必须必须要会！

- **int corePoolSize - 核心线程数**
 - 规范线程池中，最多保留多少个空闲得线程个数。
- **int maximumPoolSize - 最大线程数**
 - 规范线程池中，最多可以有多少个线程。
- **long keepAliveTime - 空闲时间**
 - 指定工作线程在线程池中最多等待多久新投递过来的任务。
 - 如果工作线程个数，没有超过corePoolSize，不会涉及这个属性。
(allowCoreThreadTimeOut默认为false的情况)
- **TimeUnit unit - 空闲时间的单位**
- **BlockingQueue workQueue - 工作队列**
 - 当工作线程数达到了corePoolSize时，再投递过来的任务，就会扔到工作队列
- **ThreadFactory threadFactory - 线程工厂**
 - 线程池创建线程的方式，就是基于线程工厂创建的
- **RejectedExecutionHandler handler - 拒绝策略（默认抛异常）**
 - 当线程个数达到了 maximumPoolSize，并且工作队列的任务达到了队列长度的限制，就会触发拒绝策略，不处理当前任务

八、项目中线程池的使用场景？项目中有用到多线程嘛？（画龙科技）

无论如何，给自己的项目搞一个使用多线程的场景！！

哪怕没有，润色也要润一个！！

1、考虑一下有没有处理数据体量比较大的场景，就可以将这些数据做切分，多线程并行处理

比如，上游基于Kafka、导入Excel数据、查询三方.....给你甩过来了上万条数据，这里可以将上万条数据基于5000条一坨拆分投递给线程池处理，由线程池并行得去做数据的校验，清洗、处理等等操作，最终批量落库.....

2、也可以考虑业务线比较长的情况，**优先只让核心业务串行向下执行，快速给客户响应**，一些非核心的业务，可以基于线程池去异步处理。

3、也可以考虑业务中涉及到了多次网络IO的情况，必须需要查询三方获取数据，查询其他服务获取数据，查询数据库获取数据等等，多次网络IO如果串行执行，性能会收到影响，如果业务里得多次网络IO，不涉及到前后依赖的情况，完全可以让这些涉及网络IO的操作并行执行。原生的方式，可以 **ThreadPoolExecutor + CountdownLatch**，也可以基于 **CompletableFuture** 提供的 **allOf**处理。

Ps：必须必须扣到业务上，操作的是啥业务，啥数据，一定要扣上！

可能聊到线程池落地后，会涉及到一些线程池参数配置的问题，看一下，下面这节课就足够了。

<https://www.mashibing.com/live/2809?r=2565&courseVersionId=3758>